

WEEKLY ENGINEERING PROGRESS REPORT

Node: Spatial Telemetry Engine (spatial-telemetry-engine)

Engineer Details: Name: Aarav Yadav
Role: AI/ML Software Engineer Intern
targeting Spatial Data & Computer Vision
with Gen AI

1. Accomplishments This Week (The Spatial Infrastructure Pivot)

This week, the core engineering focus shifted from generic document tokenization to building a production-grade, localized data infrastructure platform optimized for multi-dimensional robotics datasets. The primary development track moved away from text processing constraints to directly handle high-leverage spatial computing variables.

Milestone A: Baseline Document RAG Validation

- **System Action:** Configured and validated a fully local, asynchronous FastAPI backend running a text-based Retrieval-Augmented Generation (RAG) framework.
- **Mechanism:** Engineered a character-splitting token utility to partition document arrays into overlapping 500-word windows.
- **Database & Model Coupling:** Orchestrated socket handshakes with a local PostgreSQL instance clustered with the pgvector extension and automated background daemon process control routines for local nomic-embed-text and llama3.2 execution layers.

Milestone B: Architectural Refactor to Spatial Telemetry Engine

- **The Pivot Goal:** Completely stripped out text-document limits to process high-leverage spatial variables including 3D bounding boxes, camera calibration matrices, and vehicle velocity profiles.
- **Structural Pydantic Schemas:** Implemented dual-layer strict model contracts (BoundingBox3D and SpatialFramePayload) enforcing strict dimensional constraints (exactly 3 items for [X, Y, Z] vectors), preventing memory geometry unalignments.
- **Dense Topology Serialization:** Wrote the `serialize_spatial_frame` routine to flatten multi-dimensional matrix inputs into dense structural semantic strings by computing the **Euclidean Norm** of the ego-velocity vector ($\|v\| = \sqrt{V_x^2 + V_y^2 + V_z^2}$) and radial distance calculations to target tracks.
- **Low-Latency Database Clustering:** Mutated the database storage schema into a permanent `spatial_scene_store` table, saving high-density jsonb structural tracking logs alongside a 768-dimensional vector space.
- **Deterministic Simulation Seeding Query:** Refactored the `/query/scenario` endpoint to execute lightning-fast geometric **Cosine Distance** searches ($\Leftrightarrow$) inside PostgreSQL to isolate matching frame sequences in under 5ms. This passes raw coordinate arrays to llama3.2 to synthesize exact **4D state vector initialization parameters** ($x = [X, Y, \dot{X}, \dot{Y}]^T$) for downstream C++ Kalman filters.

2. Today's Execution Targets (Closing the Host Loop)

Today's sprint focus is on completing the interface and simulation layers on the local host machine to see this engine move and breathe.

1. Build the Interactive User Interface (`ui.py`)

- **Action:** Implement a clean, responsive Streamlit dashboard interface layer.
- **Features:** Construct a **Spatial Telemetry Ingestion Workspace** to visually upload/paste tracking JSON coordinates, and a **Conversational Scenario Workspace** to test natural language boundary constraint queries in real-time.

2. Build the Simulation Mock Data Feeder (`test_ingest.py`)

- **Action:** Program a specialized Python loop script to act as the front-end perception layer simulation.
- **Features:** Generate simulated synthetic time-series frames containing moving cars, varying camera extrinsics $[R|t]$, and transient occlusion flags (e.g., simulating a 500ms blind spot), then pipe them as automated HTTP POST streams directly into the backend `/ingest/spatial` endpoint to verify indexing integrity and populate `aarav_vector_db`.

3. Next Week's Deterministic Runway

Next week is focused on validation testing, performance optimization, and shifting toward pipeline scale integration.

- **Task 1: Complete Dataset Bridge Testing (nuScenes/Waymo Parsing)** Write a small utility converter script that reads actual raw JSON/binary log data metadata from open-source autonomous vehicle datasets like nuScenes or Waymo. This will feed real-world sensor layouts through the `SpatialTelemetryEngine` to stress-test indexing times under larger data arrays.
- **Task 2: Performance Profiling & Latency Optimization** Profile the end-to-end ingestion loop latency. Benchmark the overhead of the Pydantic matrix verification layer against raw binary header blocks to minimize the pipeline processing time frame.
- **Task 3: Sequence Tracking Optimization** Refactor the database queries to evaluate temporal relationships across multiple contiguous frames, allowing users to isolate complex, multi-frame maneuvers (e.g., aggressive cut-ins or dual-lane transitions).

Report compiled securely on Arch Linux Host PC Maomao